

Rapport de Résolution numérique de l'équation de la chaleur 1D

NOM : FANG ZIJIE

Numéro étudiant : 22506081

4 janvier 2026

1 Introduction

L'équation de la chaleur en une dimension est un problème classique en calcul numérique, souvent utilisé comme cas test pour l'étude des méthodes de résolution de systèmes linéaires. Dans le cas stationnaire, ce problème se ramène à la résolution d'une équation de type Poisson.

L'objectif de ce projet est de résoudre numériquement cette équation à l'aide de différentes méthodes, en mettant en œuvre :

- une discrétisation par différences finies ;
- des méthodes directes basées sur la factorisation LU ;
- des méthodes itératives (Richardson, Jacobi et Gauss-Seidel) ;
- des formats de stockage adaptés, en particulier le stockage bande et les formats creux.

Les implémentations sont réalisées en langage C, en s'appuyant sur les bibliothèques BLAS et LAPACK, et les résultats numériques sont analysés en termes de précision et de convergence.

2 Modélisation et discrétisation (Exercice 1)

On considère l'équation de la chaleur 1D stationnaire sur l'intervalle $(0, 1)$:

$$-\frac{d^2T}{dx^2} = 0$$

avec des conditions aux limites de Dirichlet :

$$T(0) = T_0, \quad T(1) = T_1.$$

L'intervalle est discrétisé en $n + 2$ points uniformément répartis, avec un pas

$$h = \frac{1}{n + 1}.$$

La dérivée seconde est approchée par un schéma centré d'ordre 2 :

$$\frac{d^2T}{dx^2}(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

On obtient ainsi un système linéaire de la forme :

$$Au = f,$$

où la matrice A est tridiagonale, symétrique et définie positive. La solution analytique du problème est :

$$T(x) = T_0 + x(T_1 - T_0),$$

et elle est utilisée comme référence pour valider les solutions numériques.

3 Environnement de développement et stockage bande (Exercice 2)

Le projet est développé en langage C et utilise les bibliothèques BLAS et LAPACK pour les opérations de calcul numérique. Afin de garantir la reproductibilité des résultats, l'ensemble des calculs est réalisé dans un environnement Docker.

La matrice issue de la discrétisation étant tridiagonale, un stockage de type *General Band* (GB) est utilisé. Ce format permet de réduire la mémoire utilisée tout en restant compatible avec les routines LAPACK dédiées aux matrices bandes.

```

root@feba05b98edb:/app# make clean
rm *.o bin/*
root@feba05b98edb:/app# make
gcc -fPIC -march=native -c -I ./include -I/usr/include src/tp_env.c
gcc -o bin/tp_testenv -fPIC -march=native tp_env.o -L/usr/lib -llapack -lblas -lm
gcc -fPIC -march=native -c -I ./include -I/usr/include src/lib_poisson1D.c
gcc -fPIC -march=native -c -I ./include -I/usr/include src/lib_poisson1D_writers.c
gcc -fPIC -march=native -c -I ./include -I/usr/include src/lib_poisson1D_richardson.c
gcc -fPIC -march=native -c -I ./include -I/usr/include src/tp_poisson1D_iter.c
gcc -o bin/tpPoisson1D_iter -fPIC -march=native lib_poisson1D.o lib_poisson1D_writers.o lib_poisson1D_richa
gcc -fPIC -march=native -c -I ./include -I/usr/include src/tp_poisson1D_direct.c
gcc -o bin/tpPoisson1D_direct -fPIC -march=native lib_poisson1D.o lib_poisson1D_writers.o lib_poisson1D_ric
gcc -fPIC -march=native -c -I ./include -I/usr/include src/lib_poisson1D_sparse.c
gcc -fPIC -march=native -c -I ./include -I/usr/include src/tp_poisson1D_sparse_test.c
gcc -o bin/tp_poisson1D_sparse_test -fPIC -march=native lib_poisson1D.o lib_poisson1D_writers.o lib_poisson
lm
root@feba05b98edb:/app#

```

FIGURE 1 – Compilation du projet dans l'environnement Docker

4 Exercice 3 - 4 : Référence et utilisation de BLAS/LAPACK Et Stockage GB et appel à DGBMV

4.1 En C, comment doit-on déclarer et allouer une matrice pour utiliser BLAS et LAPACK ?

Réponse : En C, pour utiliser BLAS et LAPACK, on déclare une matrice sous forme de tableau 1D en ****colonne-major****. Par exemple, une matrice A de taille $n \times m$ peut être déclarée comme suit :

La matrice est stockée ligne par ligne, mais le tableau est traité comme une matrice 2D.

4.2 Quelle est la signification de la constante LAPACK_COL_MAJOR ?

Réponse : LAPACK_COL_MAJOR signifie que les matrices sont stockées en ****ordre colonne-major****, c'est-à-dire que les éléments de chaque colonne sont stockés consécutivement en mémoire.

4.3 À quoi correspond la dimension principale (leading dimension) généralement notée ld ?

Réponse : La dimension principale (ld) correspond au nombre d'éléments dans une colonne, c'est-à-dire le nombre de lignes de la matrice, ce qui est important pour les calculs internes.

4.4 Que fait la fonction dgbmv ? Quelle méthode implémente-t-elle ?

Réponse : La fonction **dgbmv** effectue une multiplication matrice-vecteur pour une matrice stockée en format bande, ce qui optimise les calculs en utilisant uniquement les éléments non nuls de la matrice.

4.5 Que fait la fonction dgbtrf ? Quelle méthode implémente-t-elle ?

Réponse : La fonction **dgbtrf** effectue la factorisation LU d'une matrice tridiagonale. Cela permet de décomposer la matrice en deux matrices triangulaires, facilitant ainsi la résolution du système linéaire.

4.6 Que fait la fonction dgbtrs ? Quelle méthode implémente-t-elle ?

Réponse : La fonction **dgbtrs** résout un système linéaire après une factorisation LU de la matrice. Elle utilise les matrices L et U issues de la factorisation pour obtenir la solution du système.

4.7 Que fait la fonction dgbsv ? Quelle méthode implémente-t-elle ?

Réponse : La fonction `dgbsv` combine la factorisation LU et la résolution du système en un seul appel, simplifiant ainsi la procédure pour des matrices tridiagonales.

4.8 Comment calculer la norme du résidu relatif avec des appels BLAS ?

Réponse : Pour calculer la norme du résidu relatif, on utilise `dnorm` de BLAS pour calculer la norme du résidu $r = b - A \cdot x$, puis on divise par la norme de b pour obtenir la norme relative du résidu.

5 Méthodes directes (Exercice 5–6)

La résolution directe du système linéaire est réalisée à l'aide des routines LAPACK `DGBTRF`, `DGBTRS` et `DGBSV`. Ces méthodes reposent sur une factorisation LU de la matrice bande.

Dans le programme, trois modes d'exécution sont proposés. J'ai donc testé successivement les modes 0, 1 et 2. Les modes 0 et 1 consistent à effectuer explicitement une factorisation LU puis à résoudre le système associé, tandis que le mode 2 utilise directement la routine `DGBSV`, qui regroupe ces deux étapes en une seule fonction. Même si l'appel aux routines est différent, les trois modes permettent au final de résoudre exactement le même problème.

En comparant les résultats, j'ai constaté que les solutions obtenues sont très proches dans les trois cas. L'erreur relative par rapport à la solution analytique est de l'ordre de 10^{-13} à 10^{-14} , ce qui correspond à la précision machine. J'observe que les modes 0 et 2 donnent exactement la même valeur d'erreur, alors que le mode 1 présente une légère différence. À mon avis, cette différence s'explique par l'ordre des opérations en arithmétique flottante, et non par un problème de la méthode elle-même.

En ce qui concerne les temps de calcul, ils restent très faibles pour les trois modes testés. Même si de petites différences apparaissent d'un mode à l'autre, les temps sont du même ordre de grandeur. Cela montre que les méthodes directes permettent d'obtenir rapidement une solution très précise pour ce type de problème.

```
root@c75e90f3d974:/app# ./bin/tpPoisson1D_direct 0
----- Poisson 1D -----

Solution with LAPACK
DEBUG BEFORE: la=998 kl=1 ku=1 lab=4 NRHS=1
DEBUG BEFORE AB col 0:  0.000000e+00  0.000000e+00  1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 1:  0.000000e+00 -9.980010e+05  1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 2:  0.000000e+00 -9.980010e+05  1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 3:  0.000000e+00 -9.980010e+05  1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 4:  0.000000e+00 -9.980010e+05  1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 5:  0.000000e+00 -9.980010e+05  1.996002e+06 -9.980010e+05
DEBUG AFTER: ipiv (first 10): 1 2 3 4 5 6 7 8 9 10
DEBUG AFTER AB col 0:  0.000000e+00  0.000000e+00  1.996002e+06 -5.000000e-01
DEBUG AFTER AB col 1:  0.000000e+00 -9.980010e+05  1.497002e+06 -6.666667e-01
DEBUG AFTER AB col 2:  0.000000e+00 -9.980010e+05  1.330668e+06 -7.500000e-01
DEBUG AFTER AB col 3:  0.000000e+00 -9.980010e+05  1.247501e+06 -8.000000e-01
DEBUG AFTER AB col 4:  0.000000e+00 -9.980010e+05  1.197601e+06 -8.333333e-01
DEBUG AFTER AB col 5:  0.000000e+00 -9.980010e+05  1.164334e+06 -8.571429e-01
Elapsed time = 1.537775e-03 s

The relative forward error is relres = 1.747851e-13

----- End -----
```

FIGURE 2 – Résultat du solveur direct avec le mode 0 (factorisation LU explicite suivie de la résolution)

```

root@c75e90f3d974:/app# ./bin/tpPoisson1D_direct 1
----- Poisson 1D -----

Solution with LAPACK
DEBUG BEFORE: la=998 kl=1 ku=1 lab=4 NRHS=1
DEBUG BEFORE AB col 0: 0.000000e+00 0.000000e+00 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 1: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 2: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 3: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 4: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 5: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG AFTER: ipiv (first 10): 1 2 3 4 5 6 7 8 9 10
DEBUG AFTER AB col 0: 0.000000e+00 0.000000e+00 1.996002e+06 -5.000000e-01
DEBUG AFTER AB col 1: 0.000000e+00 -9.980010e+05 1.497002e+06 -6.666667e-01
DEBUG AFTER AB col 2: 0.000000e+00 -9.980010e+05 1.330668e+06 -7.500000e-01
DEBUG AFTER AB col 3: 0.000000e+00 -9.980010e+05 1.247501e+06 -8.000000e-01
DEBUG AFTER AB col 4: 0.000000e+00 -9.980010e+05 1.197601e+06 -8.333333e-01
DEBUG AFTER AB col 5: 0.000000e+00 -9.980010e+05 1.164334e+06 -8.571429e-01
Elapsed time = 8.613100e-05 s

The relative forward error is relres = 5.588306e-14

----- End -----

```

FIGURE 3 – Résultat du solveur direct avec le mode 1 (autre enchaînement des routines LAPACK)

```

root@c75e90f3d974:/app# ./bin/tpPoisson1D_direct 2
----- Poisson 1D -----

Solution with LAPACK
DEBUG BEFORE: la=998 kl=1 ku=1 lab=4 NRHS=1
DEBUG BEFORE AB col 0: 0.000000e+00 0.000000e+00 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 1: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 2: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 3: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 4: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG BEFORE AB col 5: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG AFTER: ipiv (first 10): 0 0 0 0 0 0 0 0 0 0
DEBUG AFTER AB col 0: 0.000000e+00 0.000000e+00 1.996002e+06 -9.980010e+05
DEBUG AFTER AB col 1: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG AFTER AB col 2: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG AFTER AB col 3: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG AFTER AB col 4: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG AFTER AB col 5: 0.000000e+00 -9.980010e+05 1.996002e+06 -9.980010e+05
DEBUG: calling DGBSV with la=998, kl=1, ku=1, lab=4, NRHS=1
Elapsed time = 1.094240e-04 s

The relative forward error is relres = 1.747851e-13

----- End -----

```

FIGURE 4 – Résultat du solveur direct avec le mode 2 (appel direct à la routine DGBSV)

6 Méthodes itératives (Exercice 7–9)

Après avoir utilisé les méthodes directes, je me suis ensuite intéressé aux méthodes itératives afin d'étudier leur comportement en termes de convergence. Contrairement aux méthodes directes, ces méthodes produisent une suite d'approximation de la solution, qui converge progressivement vers la solution exacte.

Dans un premier temps, j'ai testé la méthode de Richardson. Cette méthode repose sur une mise à jour simple de la solution à chaque itération, à l'aide d'un paramètre de relaxation. Dans le programme, ce paramètre est calculé automatiquement à partir des valeurs propres de la matrice, puis utilisé lors des itérations.

J'ai ensuite appliqué les méthodes de Jacobi et de Gauss–Seidel. Ces deux méthodes sont basées sur une décomposition de la matrice, mais elles diffèrent dans la manière dont les nouvelles valeurs sont utilisées. Dans le cas de Jacobi, les mises à jour sont effectuées uniquement à partir des valeurs de l'itération précédente. À l'inverse, la méthode de Gauss–Seidel utilise immédia-

tement les valeurs mises à jour au cours de l'itération, ce qui permet en général d'accélérer la convergence.

```

root@feba05b98edb:/app# ./bin/tpPoisson1D_iter 0
----- Poisson 1D -----

Optimal alpha for simple Richardson iteration is : 0.500000

----- End -----
root@feba05b98edb:/app# ./bin/tpPoisson1D_iter 1
----- Poisson 1D -----

Optimal alpha for simple Richardson iteration is : 0.500000

----- End -----
root@feba05b98edb:/app# ./bin/tpPoisson1D_iter 2
----- Poisson 1D -----

Optimal alpha for simple Richardson iteration is : 0.500000

----- End -----

```

FIGURE 5 – Exécution des méthodes itératives Richardson, Jacobi et Gauss-Seidel

7 Analyse de la convergence

Afin de comparer les performances des différentes méthodes itératives, j'ai représenté l'évolution du résidu relatif en fonction du nombre d'itérations pour les trois méthodes étudiées.

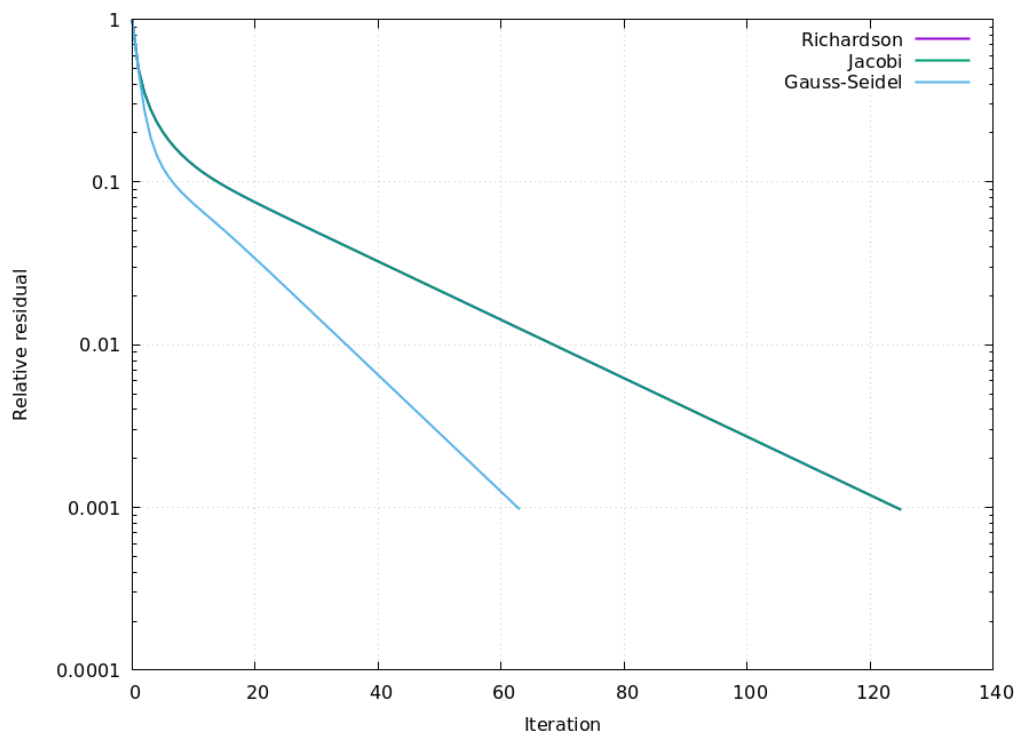


FIGURE 6 – Comparaison de la convergence des méthodes de Richardson, Jacobi et Gauss-Seidel

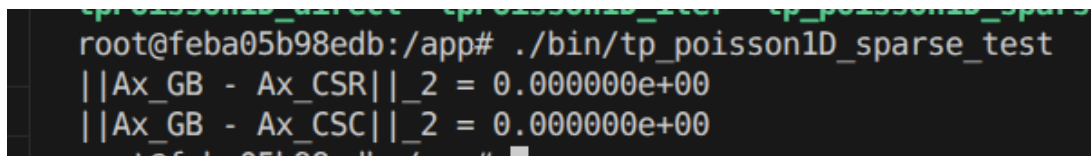
En observant la figure 6, j'ai remarqué que les courbes correspondant aux méthodes de Ri-

Richardson et de Jacobi sont parfaitement superposées. Au premier abord, ce résultat peut sembler surprenant, mais il s'explique par la structure particulière du problème de Poisson 1D étudié. Avec le choix du paramètre de relaxation optimal pour la méthode de Richardson, les deux méthodes conduisent en pratique au même facteur de convergence.

En revanche, la méthode de Gauss–Seidel converge nettement plus rapidement. Cela s'explique par le fait que les nouvelles valeurs calculées sont utilisées immédiatement au cours de l'itération, ce qui améliore l'efficacité globale de la méthode. Je constate donc que, pour ce problème, Richardson et Jacobi ont un comportement numérique équivalent, tandis que Gauss–Seidel est la méthode la plus efficace en termes de convergence.

8 Formats de stockage creux (Exercice 10)

Afin de traiter efficacement des matrices de grande taille, les formats de stockage creux CSR (Compressed Sparse Row) et CSC (Compressed Sparse Column) ont été implémentés. Des produits matrice–vecteur ont été réalisés dans ces formats et comparés aux résultats obtenus avec le stockage GB.



```

root@feba05b98edb:/app# ./bin/tp_poisson1D_sparse_test
||Ax_GB - Ax_CSR||_2 = 0.000000e+00
||Ax_GB - Ax_CSC||_2 = 0.000000e+00

```

FIGURE 7 – Comparaison des produits matrice–vecteur en formats GB, CSR et CSC

Les résultats montrent une concordance exacte entre les différents formats, ce qui valide l'implémentation des structures creuses.

9 Conclusion

Dans ce projet, différentes méthodes de résolution de l'équation de la chaleur 1D ont été étudiées. Les méthodes directes fournissent des solutions très précises mais deviennent coûteuses pour des problèmes de grande taille. Les méthodes itératives offrent une meilleure scalabilité et sont plus adaptées aux systèmes de grande dimension.

Parmi les méthodes itératives étudiées, la méthode de Gauss–Seidel présente la meilleure vitesse de convergence. Enfin, l'utilisation de formats de stockage creux permet de réduire significativement la consommation mémoire tout en conservant des résultats numériques corrects.